FAISS, ChromaDB and Milvus

Detailed Professional Notes on Vector Search, Indexing, RAG Integration and Technology Selection

Coverage	Concepts, architecture, index types, trade-offs, examples, decision guidance, revision summary, interview questions and brainstorming answers.
Audience	AI engineers, solution architects, developers and technical consultants designing semantic search or RAG solutions.

Core idea: FAISS is a high-performance vector-search library. ChromaDB is an AI-focused vector database that adds persistence and metadata management. Milvus is designed for distributed, production-scale vector workloads.

Contents

1. Executive summary
2. Vector search foundations
3. FAISS
4. ChromaDB
5. Detailed comparison
6. FAISS index types: Flat, IVF, LSH and HNSW
7. Milvus and extending FAISS capabilities
8. LangChain, LlamaIndex and RAG
9. Selection framework and practical scenarios
10. Risks, design considerations and best practices
11. Quick revision summary
12. Interview questions and answers
13. Brainstorming questions and model answers

1. Executive Summary

Vector search retrieves items according to similarity in an embedding space rather than relying only on exact keyword matching. This capability underpins semantic search, recommendation systems, document retrieval and Retrieval-Augmented Generation (RAG).

Technology	Category	Primary strength	Typical fit
FAISS	Vector-search library	Low-level control and optimized similarity search	Local or single-node performance-focused implementations
ChromaDB	AI-focused vector database	Simple persistence, metadata and framework integration	Prototypes, learning projects and metadata-rich RAG
Milvus	Distributed vector database	Scalability, metadata filtering and production operations	Large, production-grade vector-search platforms

The key architectural decision is not simply which product is fastest. It is whether the solution needs a search library or a managed data layer with persistence, metadata, filtering, APIs, operational controls and horizontal scale.

2. Vector Search Foundations

An embedding model converts text, images or other content into numerical vectors. Semantically similar items are positioned close to one another in the vector space. A query is embedded using the same compatible model, and the system returns the nearest vectors according to a selected similarity measure.

Key terminology

- **Vector or embedding:** A numerical representation of content.
- **Vector store:** A storage and retrieval layer for embeddings.
- **Nearest-neighbor search:** Retrieval of the vectors closest to a query vector.
- **Exact search:** Evaluates all candidates and provides complete recall for the selected metric.
- **Approximate nearest-neighbor (ANN) search:** Reduces the candidate space to improve speed, accepting a possible loss in recall.
- **Top-k:** The number of nearest results returned.
- **Metadata:** Attributes associated with a vector, such as source, category, date, branch or document type.

Common similarity measures

Measure	Interpretation	Design note
Euclidean distance (L2)	Geometric distance between vectors	Smaller distance means greater similarity.
Dot product	Alignment and magnitude interaction	Larger value normally indicates greater similarity.
Cosine similarity	Angular similarity independent of magnitude	Often used with normalized text embeddings.

3. FAISS

FAISS, short for Facebook AI Similarity Search, is a library developed by Meta for efficient similarity search over dense vectors. It can run on CPU or GPU and exposes multiple index structures so developers can tune speed, memory consumption, build time and recall.

How FAISS is used

- Create embeddings using an embedding model.
- Choose and configure a FAISS index.
- Add vectors to the index.
- Embed the incoming query using the same model.
- Search for the top-k nearest vectors.
- Map returned vector identifiers back to application records or documents.

Strengths

- High-performance nearest-neighbor search.
- CPU and GPU execution options.
- Broad selection of exact and approximate indexes.
- Fine-grained control over index behavior and performance tuning.
- Suitable as an embedded component inside a custom application.

Limitations

- It is not, by itself, a complete database service.
- Metadata storage and filtering require an external design or wrapper.
- Application teams must implement persistence, identifier mapping, APIs, access control, backup and operational monitoring as needed.
- Native distributed scaling is not the default FAISS operating model.

Practical interpretation

FAISS is appropriate when the team wants to own the data layer and search pipeline, needs low-level index control, or wants optimized local/GPU search. Its simplicity at the library level becomes additional engineering responsibility at the platform level.

4. ChromaDB

ChromaDB is an AI-oriented vector database used to store embeddings, documents and metadata. It reduces the amount of infrastructure code required for common RAG and semantic-search experiments and integrates with frameworks such as LangChain and LlamaIndex.

Typical stored record

Element	Example	Purpose
ID	doc-001-chunk-04	Stable application reference
Embedding	[0.12, -0.44, ...]	Used for similarity retrieval
Document	Chunk text	Provides context to the application or LLM
Metadata	source=manual; topic=ATM	Supports filtering and traceability

Strengths

- Easy setup for AI development and prototypes.
- Built-in handling of embeddings, documents, identifiers and metadata.
- Metadata-based filtering improves retrieval precision.
- Convenient integration with RAG frameworks.
- Can be used locally and through a service-oriented deployment pattern.

Trade-offs

- Provides less index-level flexibility than FAISS.
- Teams should validate scale, concurrency, durability and operational requirements before choosing it for a critical production workload.
- Ease of use does not remove the need for data lifecycle, security, backup and embedding-version controls.

5. Detailed Comparison

Dimension	FAISS	ChromaDB	Milvus
Product type	Library	Vector database	Distributed vector database
Primary goal	Optimized similarity search	Developer-friendly AI data layer	Production-scale vector platform
Index control	Extensive	More opinionated	Multiple production-oriented choices
Metadata	External implementation	Built in	Built in
Filtering	Application-managed	Metadata filtering	Scalar/metadata filtering and hybrid patterns
Persistence	Developer-managed	Built in	Built in
GPU usage	Supported by FAISS variants	Not the core value proposition	Deployment/index dependent
Distributed scale	Not native	Deployment capability should be validated for the target version	Core production objective
Operational effort	Application engineering required	Lower for prototypes	Higher platform operations, but richer scale features

6. FAISS Index Types

Index selection determines the balance among recall, query latency, memory, index-build time and operational complexity. Benchmarking must use representative embeddings and queries because results vary by vector dimensionality, distribution and dataset size.

6.1 Flat Index

A Flat index performs exhaustive search. The query is compared with every stored vector using a configured distance or similarity measure. The results are ordered and the nearest k vectors are returned.

- **Accuracy:** Exact for the selected metric.
- **Speed:** Query cost grows with the number of vectors.
- **Training:** No index training is required.
- **Best use:** Small datasets, validation baselines and use cases where exact retrieval is mandatory.

6.2 IVF (Inverted File Index)

IVF partitions the vector space into clusters, commonly using k-means centroids. During search, the query is assigned to nearby clusters and only vectors in a selected number of those clusters are examined.

- **nlist:** Number of clusters or inverted lists.
- **nprobe:** Number of clusters examined per query. Increasing nprobe usually improves recall but increases latency.
- **Training:** A representative sample is required to train centroids.
- **Trade-off:** Faster than exhaustive search, but neighbors can be missed when relevant vectors fall into unsearched cells.

6.3 Locality-Sensitive Hashing (LSH)

LSH uses hash functions designed so that similar vectors have a higher probability of mapping to the same or nearby buckets. Search focuses on matching candidate buckets rather than the full collection.

- Useful for approximate search where hashing matches the data and distance assumptions.
- Can be memory efficient and reduce candidate comparisons.
- Accuracy and performance depend heavily on dimensionality, hash configuration and data distribution.
- It should be benchmarked rather than assumed to be the best option for every text-embedding workload.

6.4 HNSW

Hierarchical Navigable Small World is a graph-based ANN structure. Vectors are connected in layers. Sparse upper layers provide long-range navigation, while denser lower layers support increasingly precise local search.

- Search starts at an upper layer and greedily navigates toward a promising region.
- The best candidate becomes the entry point to the next, denser layer.
- The process continues until the base layer produces the final candidate set.
- HNSW commonly provides a strong speed/recall balance, with additional memory consumed by graph connections.

Important HNSW tuning concepts

Parameter	Effect
M	Controls graph connectivity. Larger values may improve recall but increase memory and build cost.
efConstruction	Controls effort during index construction. Higher values can improve graph quality at greater build cost.
efSearch	Controls search breadth. Higher values typically improve recall but increase query latency.

Index trade-off summary

Index	Search model	Recall potential	Relative speed	Memory/build profile
Flat	Exhaustive	Exact	Slower as data grows	Simple; no training
IVF	Cluster-pruned	Tunable approximate	Fast when candidate scope is limited	Requires training and parameter tuning
LSH	Hash-bucket candidates	Approximate	Workload dependent	Hash tables and configuration
HNSW	Hierarchical graph traversal	High approximate recall	Often very fast	Higher memory and graph-build effort

7. Milvus and Extending FAISS Capabilities

Milvus is a vector database designed to provide capabilities that a standalone search library does not provide by itself, including persistent collections, metadata or scalar fields, filtering, service interfaces and distributed deployment patterns. FAISS-related indexing techniques can be used within broader vector-database architectures, while the database supplies the surrounding production data layer.

Hybrid query example

Business request: Find products semantically similar to the query, but only where category is "accessories", price is below 50 and stock status is available. The vector component ranks similarity; scalar or metadata filters enforce business constraints.

When Milvus becomes relevant

- The vector corpus exceeds the comfortable capacity of one application node.
- Multiple services or teams require a centrally operated vector platform.
- Persistence, metadata filters, high availability and distributed processing are mandatory.
- The organization can support the additional operational complexity of a production data platform.

8. LangChain, LlamaIndex and RAG

LangChain and LlamaIndex are orchestration frameworks. They can connect document loaders, text splitters, embedding models, vector stores, retrievers and language models. They do not eliminate the need to choose an appropriate vector-store architecture.

Typical RAG flow

Stage	Activity	Key control
1. Ingestion	Load source documents	Source authorization and traceability
2. Chunking	Split content into retrievable units	Chunk size, overlap and semantic boundaries
3. Embedding	Generate vectors	Model and version consistency
4. Storage	Save vector, text and metadata	Schema, IDs, retention and security
5. Retrieval	Search and filter candidates	Top-k, recall and metadata constraints
6. Reranking	Reorder candidates if required	Relevance versus added latency
7. Generation	Provide context to the LLM	Prompt controls and citation behavior
8. Evaluation	Measure retrieval and answer quality	Golden queries, recall, faithfulness and latency

Critical consistency rule

Documents and queries must be embedded using compatible models and preprocessing. Changing the embedding model normally requires controlled re-embedding and index migration because vectors from unrelated embedding spaces are not directly comparable.

9. Selection Framework and Practical Scenarios

Requirement	Recommended starting point	Reason
Local experiment with complete index control	FAISS	Minimal abstraction and broad index options
Personal document assistant or learning prototype	ChromaDB	Rapid setup, persistence and metadata
Accuracy baseline for ANN testing	FAISS Flat	Provides exact-search comparison
Large enterprise knowledge platform	Milvus	Designed for scalable data services and filtering
GPU-accelerated local similarity service	FAISS	Direct GPU-oriented search options
Metadata-rich departmental RAG prototype	ChromaDB	Simple record and filter model
Multi-tenant production vector service	Milvus after architecture validation	Better alignment with distributed platform requirements

Decision questions

- How many vectors are expected now and after growth?
- What are the embedding dimension and storage footprint?
- Is metadata filtering mandatory, and how complex are the filters?
- What query latency, throughput and recall targets are required?
- Is exact search necessary, or is ANN acceptable?
- Does the workload require GPU acceleration?
- Is horizontal scaling or high availability required?
- Who will operate backups, upgrades, monitoring and incident recovery?

- Are tenant isolation, encryption and audit controls required?
- How will embedding-model upgrades and re-indexing be managed?

10. Risks, Design Considerations and Best Practices

- **Benchmark with real data:** Synthetic results may not represent enterprise documents, logs or multilingual content.
- **Measure recall and latency together:** A fast search that misses relevant context can reduce RAG answer quality.
- **Keep an exact baseline:** Use a Flat index on a manageable evaluation set to compare ANN recall.
- **Version the embedding pipeline:** Record model, preprocessing, chunking and index configuration.
- **Preserve source traceability:** Store document ID, chunk ID, source path, version and authorization metadata.
- **Apply security before retrieval:** Do not depend on the LLM to remove unauthorized content after retrieval.
- **Plan deletion and re-indexing:** Enterprise sources change; stale vectors can create incorrect answers.
- **Monitor end-to-end quality:** Retrieval relevance, generation faithfulness, latency and cost should be evaluated together.
- **Validate product claims against the selected version:** Deployment and index capabilities evolve, so architecture should follow tested documentation and proof-of-concept evidence.

11. Quick Revision Summary

Topic	Remember
FAISS	A high-performance vector-search library with extensive index control; surrounding database capabilities are application responsibilities.
ChromaDB	A developer-friendly vector database for embeddings, documents and metadata; useful for rapid RAG development.
Milvus	A production-oriented vector database for persistent, metadata-aware and distributed vector workloads.
Flat	Exact exhaustive search; best accuracy baseline but query cost grows with collection size.
IVF	Clusters vectors and searches selected cells; tune nlist and nprobe.
LSH	Uses similarity-preserving hashing to reduce candidates; highly workload dependent.
HNSW	Navigates a layered proximity graph; strong speed/recall trade-off with additional memory.
RAG	Retrieval quality depends on chunking, embeddings, filters, index tuning and evaluation, not only on the LLM.

Memory aid

FAISS = control and performance.
ChromaDB = simplicity and rapid AI development.
Milvus = scalable production vector platform.
Flat = exact. IVF = clustered. LSH = hashed. HNSW = graph-based.

12. Interview Questions and Answers

1. What is FAISS?

FAISS is a vector-similarity search library developed by Meta. It supports CPU and GPU execution and offers multiple exact and approximate index structures for nearest-neighbor retrieval.

2. Is FAISS a vector database?

No. It is primarily a search library. A complete solution must separately address persistence, metadata, filtering, APIs, authorization, backup and distributed operations where required.

3. What distinguishes ChromaDB from FAISS?

ChromaDB provides a database-oriented abstraction for embeddings, documents, identifiers and metadata. FAISS exposes lower-level index functionality and gives the developer more direct control over vector-search behavior.

4. What is a Flat index?

A Flat index compares the query with every stored vector. It provides exact results for the configured metric and is useful for small datasets or as a recall benchmark.

5. How does IVF improve search performance?

IVF clusters vectors into inverted lists. At query time, it searches only selected nearby lists, reducing comparisons. The nprobe setting controls the speed-versus-recall trade-off.

6. What is locality-sensitive hashing?

LSH uses hash functions designed to place similar vectors in the same or nearby buckets with high probability, allowing the search to evaluate a reduced candidate set.

7. Explain HNSW simply.

HNSW builds a multilayer proximity graph. Sparse upper layers move quickly toward the relevant region, while dense lower layers refine the search among nearby vectors.

8. Why can HNSW use more memory?

Each vector stores graph connections to neighboring vectors. Greater connectivity can improve navigation and recall, but increases memory and construction cost.

9. Why is metadata important in RAG?

Metadata supports source tracking, filtering, access enforcement and contextual constraints such as document type, business unit, language or effective date.

10. When would you use Milvus?

Use it when the solution requires a centrally operated, persistent and scalable vector database with metadata filtering and production platform capabilities.

11. What is the difference between exact and approximate search?

Exact search guarantees the nearest results for the selected metric by evaluating the full required search space. ANN methods reduce candidate evaluation for speed and may miss some true neighbors.

12. How should an index be selected?

Evaluate dataset scale, dimensionality, recall target, latency, throughput, memory, build time, update pattern and operational constraints. Benchmark options using representative data.

13. Why maintain an exact-search baseline?

It provides a reference result set. ANN recall can then be measured by comparing approximate results with the exact nearest neighbors.

14. What happens when the embedding model changes?

The vector space changes. A controlled migration normally requires re-embedding source content, building a new index, validating it and switching traffic safely.

15. Do LangChain or LlamaIndex replace a vector database?

No. They orchestrate components and integrations. The selected vector store still determines storage, indexing, filtering, scaling and operational characteristics.

13. Brainstorming Questions and Model Answers

These questions are designed for architecture discussions, interviews and deeper study. The model answers show the reasoning expected from an experienced practitioner.

Why might FAISS outperform a database-based solution in a local workload?

Model answer: FAISS can operate directly inside the application with minimal service and data-layer overhead. It also exposes optimized index choices and GPU support. However, the comparison must include any external metadata and persistence components added around it.

Why can a Flat index become impractical as the dataset grows?

Model answer: Each query evaluates every vector. The workload therefore grows with collection size, increasing latency or compute consumption even though retrieval remains exact.

How would you tune IVF when recall is too low?

Model answer: Increase nprobe so more clusters are searched, review the number and quality of centroids, confirm the training sample represents production data, and compare results against a Flat baseline. Every change should be measured for both recall and latency.

When could HNSW be a poor fit?

Model answer: It may be unsuitable when memory is highly constrained, indexes are rebuilt frequently, or an exact guarantee is mandatory. Frequent high-volume updates and deletion patterns should also be tested against the implementation.

Why should metadata filtering occur before or during retrieval?

Model answer: It limits candidates to records the user is permitted to access and to content relevant to the business request. This improves precision and prevents unauthorized chunks from entering the LLM context.

How would you test a vector store for an ATM support knowledge assistant?

Model answer: Build a representative corpus of manuals, incident notes and troubleshooting procedures. Define golden queries covering error codes, components and symptoms. Measure recall, source correctness, answer faithfulness, access filtering and latency. Include terminology variations and incomplete log descriptions.

What architecture would you propose for an early proof of concept that may later scale?

Model answer: Start with a clean abstraction around the vector-store interface and keep document IDs, metadata and embedding versions portable. Use ChromaDB or FAISS for the proof of concept depending on filtering and control needs, but define migration tests and operational requirements before moving to a distributed platform.

How do chunk size and index selection interact?

Model answer: Chunking controls the semantic units stored in the index. Very small chunks may lose context; very large chunks may dilute relevance. Index tuning cannot compensate for fundamentally poor chunking, so both should be evaluated together.

Why is the fastest index not always the best choice?

Model answer: The objective is useful retrieval, not latency alone. A faster index with lower recall may omit the evidence required for a correct answer, causing the downstream LLM to respond incompletely or incorrectly.

What should be recorded for a reproducible benchmark?

Model answer: Dataset version, embedding model and version, preprocessing, vector dimension, index type and parameters, hardware, query set, concurrency, warm-up procedure, top-k, recall metric, latency percentiles and memory usage.

How would you protect confidential enterprise documents in RAG?

Model answer: Enforce identity and authorization in the retrieval layer, attach access-control metadata, filter before context construction, encrypt data, isolate tenants, audit retrieval, minimize retained content and test for permission leakage.

What is a safe migration approach between vector stores?

Model answer: Create a target schema, re-embed only if required, bulk-load and validate counts and metadata, run parallel retrieval evaluation, compare recall and latency, test security filters, then perform a controlled traffic transition with rollback capability.

How should a team choose among FAISS, ChromaDB and Milvus?

Model answer: Begin with functional and operational requirements rather than popularity. FAISS fits embedded high-control search; ChromaDB fits rapid AI development and straightforward metadata use; Milvus fits a shared, scalable production platform. Validate the decision through a representative proof of concept.

Final Takeaway

The correct technology depends on the boundary of responsibility. Choose FAISS when the application should own a highly optimized search component. Choose ChromaDB when development speed, persistence and metadata convenience are the priority. Choose Milvus when the organization requires a scalable vector-data platform. In all cases, production success depends on disciplined embedding management, metadata design, security, evaluation and operations.